OMI Doctrine

The Third Collapse

The history of computing advances through the collapse of artificial distinctions.

Lisp

Lisp collapsed the distinction between program and object.

```
program ↔ object
```

The artifact of this collapse is the S-expression.

Code may be treated as data.

Data may be treated as code.

POSIX

POSIX collapsed the distinction between storage and communication.

```
storage ↔ communication
```

The artifact of this collapse is the file descriptor.

A file, pipe, terminal, device, or socket may all be accessed through the same interface.

OMI

OMI collapses the distinction between representation and interpretation.

```
representation ↔ interpretation
```

The artifact of this collapse is the rewrite surface.

The same binary source may be interpreted through multiple lawful readings without changing the source itself.

The OMI Principle

OMI begins from a single observation:

```
The binary source remains.

The interpretation changes.
```

Traditional systems treat computation as mutation.

```
input
  ↓
transformation
  ↓
output
```

OMI treats computation as reinterpretation.

```
source
  ↓
notation
  ↓
reading
  ↓
receipt
```

The source remains unchanged.

The reading rotates.

The rewrite is the computation.

Notation as Cipher

A cipher is normally understood as a mechanism for obscuring information.

OMI generalizes the concept.

A cipher is any lawful transformation of interpretation.

Likewise, a notation is any lawful declaration of interpretation.

Therefore:

```
notation as cipher
cipher as notation
```

The distinction disappears.

The same surface may be both.

The Rewrite Surface

A rewrite surface is a binary source viewed through an active notation.

Examples include:

```
ASCII
Unicode
UTF-EBCDIC
BiDi
BOM markers
frame headers
control characters
routing declarations
```

These do not necessarily change data.

They change how data is read.

OMI therefore treats them as computational operators.

Control Characters

Traditional computing describes ASCII control codes as non-printing characters.

OMI treats them as rewrite operators.

```
NUL
SOH
STX
ETX
EOT
ACK
BEL
BS
HT
LF
CR
SO
SI
```

These were historically protocol signals.

They altered the state of the receiving machine.

OMI restores that interpretation.

Control characters are operational geometry.

They are not merely text.

Rotation Instead of Mutation

OMI prefers rotation to destruction.

A shift discards information.

A rotation preserves information.

The canonical law is:

$\Delta C(x) = \text{rotl}(x,1) \oplus \text{rotl}(x,3) \oplus \text{rotr}(x,2) \oplus C$

where:

```
C = carried closure of the orbit
```

The carried closure acts as a witness of prior state.

Nothing falls off the edge.

The edge returns through the orbit.

Projective Closure

Finite state spaces possess closure.

OMI models this closure projectively.

Examples:

```
16 states
= 15 active states + 1 closure

8 states
= 7 active states + 1 closure
```

The missing state is not absent.

It is the projective center.

It is the closure through which the orbit returns.

The OMI Arithmetic

OMI does not store data.

OMI preserves versioned binary sources of truth.

The binary source becomes a rewrite surface.

The rewrite surface becomes a lawful interpretation.

The interpretation becomes a receipt.

```
source
→ notation
→ interpretation
```

```
→ validation
→ receipt
```

The OMI Invariant

The source remains.

The reading changes.

The rewrite is the computation.

The One-Line Canon

Lisp collapsed program and object.

POSIX collapsed storage and communication.

OMI collapses representation and interpretation:

the same binary surface is both notation and cipher, distinguished only by the active reading, because computation is the rewrite of interpretation rather than the mutation of data.

The Shortest Form

OMI is notation as cipher and cipher as notation.